

LEX

set app. 1 / 3

```
/* float0.1 */
%%
[+-]?[0-9]*(\.)?[0-9]+      printf("FLOAT");
%%
```

input: ab7.3c--5.4.3+d++5-
output: abFLOATc-FLOATFLOAT+d+FLOAT-

```
/* float1.1 */
%option main
digit      [0-9]
%%
[+-]?{digit}*(\.)?{digit}+    printf("FLOAT");
%%
```

salvati fisierul float0.l cu numele float1.l si
 modificati adaugand doar digit [0-9]

```
/* float2.1 */
%option main
digit      [0-9]
sign       [+ -]
float val;
%%
{sign}?{digit}*(\.)?{digit}+    {sscanf(yytext, "%f", &val);
                                printf(">%f<", val);}
```

similar salvati float2.l
 %% sa inchideti toate regulile la final
 pentru toate app. urmatoare

Input	Output
ali-7.8veli	ali>-7.800000<veli
ali--07.8veli	ali->-7.800000<veli
+3.7.5	>3.700000<>0.500000<

```
/* echo-upcase-wrods.1 */
%option main
%%
[A-Z]+[ \t\n\.\,]    printf("%s",yytext);.      /* no action
specified */
%%
```

Input Ali VELI A7, X. 12 HAMI BEY a
Output VELI X. HAMI BEY

```
/* def-in-def.1 */
%option main
alphabetic      [A-Za-z]
digit           [0-9]
alphanumeric    ({alphabetic}|{digit})
%%
{alphabetic}{alphanumeric}*  printf("Pascal variable");
\,                          printf("Comma");
\{                          printf("Left brace");
\:\=                        printf("Assignment");
```

```
/* rule-order.1 */
%option main
%%
for printf("FOR");
[a-z]+    printf("IDENTIFIER");
%%
```

=====

```

input          for count := 1 to 10
output  FOR IDENTIFIER := 1 IDENTIFIER 10

```

```

%option main
%%
[a-z]+    printf("IDENTIFIER");
for  printf("FOR");
%%

input          for count := 1 to 10
output  IDENTIFIER IDENTIFIER := 1 IDENTIFIER 10

```

```

/* sterge TAB si SPATII de la sfarsitul liniei date de intrare LITERE
urmate de spatii si tab rezultatul va fi fara spatii si tab ex.: HHH
*/
%%
[ \t]+$

/* adauga 3 la fiecare numar divizibil cu 7 de exemplu:
7    rezultatul va fi 10
5    rezultatul ramane 5
*/
%%
        int k;
[0-9]+  {
        k=atoi(yytext);
        if (k%7==0)
            printf("%d", k+3);
        else
            printf("%d",k);
        }

```

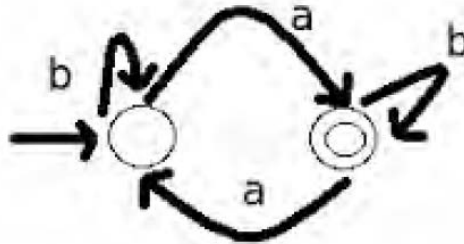
<http://2k8618.blogspot.ro/p/compiler-design-programs-lex-yacc.html>

Lex Programs

- [Implement finite automata: Odd number of a](#)
- [Line numbering](#)
- [Comment removal](#)
- [Replace multiple white spaces](#)
- [Count of a word](#)
- [Remove word](#)
- [Count of lines, words and characters](#)
- [Select lines starting with m](#)
- [Count of lines ending with com](#)
- [Count of vowels](#)

- [Count of positive numbers, negative numbers and fractions](#)
 - [Base conversion: Convert decimal numbers to hexadecimal](#)
 - [Select Lines Ending with "com"](#)
 - [Count of words starting with 'a'](#)
 - [Convert lowercase to uppercase & reverse](#)
-

Implement Finite Automata in lex-Odd no of 'a's



DFA that accept Odd no. of 'a's

```

program:
%{

%}
reg (b|ab*a)*ab*
%%
{reg}    printf("%s Accepted",yytext);
.*      printf("%s Not Accepted",yytext);

%%
main()
{
yylex();
return 0;
}
=====
Input:x.c
a
ab
aab
aba
abba
  
```

```

Output:
students@ccflab-desktop:~$ lex b.l
students@ccflab-desktop:~$ gcc lex.yy.c -lfl
students@ccflab-desktop:~$ ./a.out<x.c
a Accepted
  
```

ab Accepted
aab Not Accepted
aba Not Accepted
abba Not Accepted

Number Each Line - Lex Program - Compiler Design

Program:

```
%{  
int lineno =1;  
%}  
line .*\n  
%%  
{line} {printf("%5d %s", lineno++,yytext);}  
%%  
main()  
{  
yylex();  
return 0;  
}
```

input: compilerdesign.txt

Design of a Lexical Analyzer using Finite Automation
Design of lexical analyzer using LEX
Design of recursive descent and LL (1) parsers
Implementation of Operator precedence Parsing
Design of parser for arithmetic expressions using YACC
Design of a simple type checker
Generation of IC for arithmetic expressions
Simple code optimization strategies
Design of a code generator
Writing a simple Compiler
www.2k8618.blogspot.com

nn@linuxmint ~ \$ **lex l1.1**

nn@linuxmint ~ \$ **gcc lex.yy.c -lfl**

nn@linuxmint ~ \$ **./a.out <compilerdesign.txt**

- 1 Design of a Lexical Analyzer using Finite Automation
- 2 Design of lexical analyzer using LEX
- 3 Design of recursive descent and LL (1) parsers
- 4 Implementation of Operator precedence Parsing
- 5 Design of parser for arithmetic expressions using YACC
- 6 Design of a simple type checker
- 7 Generation of IC for arithmetic expressions
- 8 Simple code optimization strategies
- 9 Design of a code generator
- 10 Writing a simple Compiler
- 11 www.2k8618.blogspot.com

Comment Removal - Lex Program - Compiler Design

Program:

```
%{  
  
%}  
  
comment1    \/\*(.|\n)*\*/
```

```

comment2    \/\/*. *

%%

{comment1}      ;
{comment2}      ;
.|\n            ECHO;

%%
main()
{
    yylex();
    return 0;
}
=====
( input file: c1.c

#include<stdio.h>
main ()
{
    int i,n,fact=1;
    printf("Enter the number: ");
    scanf("%d",&n);           //inputing the number
    for(i=1;i<=n;i++)         /* finding factorial */
    {
        fact = fact*i;
    }
    printf("Factorial=%d\n",fact);

}
)

nn@linuxmint ~ $ lex 12.lex
nn@linuxmint ~ $ gcc lex.yy.c -lfl
nn@linuxmint ~ $ ./a.out <c1.c
#include<stdio.h>
main ()
{
    int i,n,fact=1;
    printf("Enter the number: ");
    scanf("%d",&n);
    for(i=1;i<=n;i++)
    {
        fact = fact*i;
    }
    printf("Factorial=%d\n",fact);
}

```

Replace Sequences of White Spaces by a Single Blank Character - Lex Program - Compiler Design

Elimination of Multiple Spaces ,Tabs & Emptylines

Program:

```

%{
%}

```

```

space      [ \t]
emptyline  \n
%%
{space}+   printf(" ");
{emptyline}+ printf("\n");
.          ECHO;
%%
main()
{
yylex();
}
=====

```

```

// input: sum.c
#include<stdio.h>
main()
{
//program to add two numbers
int a, b,c;
scanf("%d%d",&a,&b);
c=a+b;/* finding sum &
printing*/
printf("sum=%d",c);
}
nn@linuxmint ~ $ lex mw.l
nn@linuxmint ~ $ gcc lex.yy.c -ll
nn@linuxmint ~ $ ./a.out<sum.c
#include<stdio.h>
main()
{
//program to add two numbers
int a, b,c;
scanf("%d%d",&a,&b);
c=a+b;/* finding sum &
printing*/
printf("sum=%d",c);
}

```

Count The Number of Occurrences of a Particular Word or Letter - Lex Program - Compiler Design

Program:

```

%{
    int icount=0,factcount=0;
}%
%%
fact factcount++;
i      icount++;
(.\n)  ;

```

```
%%
main()
{
    yylex();
    printf("Count of \"fact\"= %d \nCount of letter 'i' = %d\n",factcount,icount);
    return 0;
}
```

=====

```
nn@linuxmint ~ $ lex l4.l
nn@linuxmint ~ $ gcc lex.yy.c -ll
nn@linuxmint ~ $ ./a.out<c1.c
Count of "fact"= 5
Count of letter 'i' = 17
nn@linuxmint ~ $
```

=====

// **Input file: c1.c**

```
#include<stdio.h>
main ()
{
    int i,n,fact=1;
    printf("Enter the number: ");
    scanf("%d",&n);          //inputing the number
    for(i=1;i<=n;i++)        /* finding factorial */
    {
        fact = fact*i;
    }
    printf("Factorial=%d\n",fact);
}
```

Remove All The Occurrences of a Word or Letter - Lex Program - Compiler Design Program:

```
%{

%}

%%
"printf"  ;
(.\n)    ECHO;
%%
main()
{
    yylex();
    return 0;
}
```

(input file: c1.c

```
#include<stdio.h>
main ()
```



```

{
    int i,n,fact=1;
    printf("Enter the number: ");
    scanf("%d",&n);          //inputing the number
    for(i=1;i<=n;i++)          /* finding factorial */
    {
        fact = fact*i;
    }
    printf("Factorial=%d\n",fact);
}
)
nn@linuxmint ~ $ lex l5.lex
nn@linuxmint ~ $ gcc lex.yy.c -lfl
nn@linuxmint ~ $ ./a.out <c1.c
#include<stdio.h>
main ()
{
    int i,n,fact=1;
    ("Enter the number: ");
    scanf("%d",&n);          //inputing the number
    for(i=1;i<=n;i++)          /* finding factorial */
    {
        fact = fact*i;
    }
    ("Factorial=%d\n",fact);
}

```

LEX

set app. 2 / 3

Count The Number of Lines, Words and Characters - Lex Program - Compiler Design

Program:

```
%{  
  
    int lnno=0,wordno=0,charno=0;  
}%  
word [.*.*\t]  
eol [\n]  
%%  
{word} {wordno++; charno+=yyleng;}  
{eol} {charno++;lnno++;wordno++;}  
. {charno++;}  
%%  
main()  
{  
    yylex();  
    printf("Line number= %d\n",lnno);  
    printf("Word number= %d\n",wordno);
```

```

    printf("Character number= %d\n",charno);
    return 0;
}
=====
nn@linuxmint ~ $ lex l6.l
nn@linuxmint ~ $ gcc lex.yy.c -lfl
nn@linuxmint ~ $ ./a.out <c1.c
Line number= 13
Word number= 43
Character number= 232
nn@linuxmint ~ $
( input file : c1.c
#include<stdio.h>
main ()
{
    int i,n,fact=1;
    printf("Enter the number: ");
    scanf("%d",&n);          //inputing the number
    for(i=1;i<=n;i++)        /* finding factorial */
    {
        fact = fact*i;
    }
    printf("Factorial=%d\n",fact);
}
)

```

Select Lines Starting with 'm' - Lex Program - Compiler Design Program:

```

%{
%}
reg1 ^m.*\n

%%
{reg1} {ECHO;}
. ;
%%
=====
( input file : c1.c
#include<stdio.h>
main ()
{
    int i,n,fact=1;
    printf("Enter the number: ");
    scanf("%d",&n);          //inputing the number
    for(i=1;i<=n;i++)        /* finding factorial */
    {

```

```

        fact = fact*i;
    }
    printf("Factorial=%d\n",fact);
}
)
nn@linuxmint ~ $ lex l7.lex
nn@linuxmint ~ $ gcc lex.yy.c -fl
nn@linuxmint ~ $ ./a.out <c1.c
main ()

```

Count The Number of lines ending with "com" - Lex Program - Compiler Design Program:

//Lex file: com.l

```

%{
int count=0;
%}
DIGIT [0-9]
ALPHA [a-zA-Z]
%%
({ALPHA}|{DIGIT})*com {count++;}
%%

```

```

main()
{
    yylex();
    printf("Count= %d\n",count);
    return 0;
}

```

```

=====
n@linuxmint ~ $ lex com.l
nn@linuxmint ~ $ gcc lex.yy.c -ll
nn@linuxmint ~ $ ./a.out<com.txt
www.blogspot.
www.2k.tk
www.google.
www.gmail.

```

Count= 3

Count the Vowels - Lex Program Program:

// lex file: vw.l

```
%{
int count=0;
}%
%%
[aeiouAEIOU] {count++;ECHO;}
%%
main()
{
    yylex();
    printf("\nNumber of vowels= %d\n",count);
    return 0;
}
```

=====

```
nn@linuxmint ~ $ lex vw.l
nn@linuxmint ~ $ gcc lex.yy.c -ll
nn@linuxmint ~ $ ./a.out<vw.txt
www.blogspot.com
www.2k.tk
www.google.com
www.gmail.com
```

Number of vowels= 10

Count The Positive numbers, Negative numbers & Fractions - Lex Program - Compiler Design

Aim:

Write a lex program to count the number of Positive numbers, Negative numbers & Fractions.

Program:

// lex file: a.l

```
%{
    int postiveno=0;
    int negtiveno=0;
    int positivefractions=0;
    int negativefractions=0;
}%
```

DIGIT [0-9]

```
%%
```

```
\+?{DIGIT}+          postiveno++;
-{DIGIT}+            negtiveno++;
```

```
\+?{DIGIT}*\.{DIGIT}+ positivefractions++;
-{DIGIT}*\.{DIGIT}+   negativefractions++;
```

```
. ;
%%

main()
{
    yylex();
    printf("\nNo. of positive numbers: %d",postiveno);
    printf("\nNo. of Negative numbers: %d",negtiveno);
    printf("\nNo. of Positive fractions: %d",positivefractions);
    printf("\nNo. of Negative fractions: %d\n",negativefractions);
}
```

Output:

```
nn@linuxmint ~ $ lex a.l
nn@linuxmint ~ $ gcc lex.yy.c -ll
nn@linuxmint ~ $ ./a.out<a.txt
```

```
No. of positive numbers: 2
No. of Negative numbers: 3
No. of Positive fractions: 4
No. of Negative fractions: 5
nn@linuxmint ~ $
```

```
// Input file: a.txt
+12,-123,1.1,-1.1,12,-2,-3,2.1,3.2,5.1,-5.5,-6.1,-7.7,-8.8
```

Convert Decimal Number to Hexadecimal Number in a File - Lex Program - Compiler Design

Program:

```
// lex file: conv.l

%{
    #include<stdio.h>
    int num,r,digit=0,count,pcount=0,i;
    char a[20];
}%
DIGIT [0-9]

%%

{DIGIT}+ {    num=atoi(yytext);
```

```

while(num!=0)
{
    r=num%16;
    digit='0'+r;
    if(digit>'9')
        digit+=7;
    a[count++]=digit;
    num=num/16;
}
for(i=count-1;i>=pcount;--i)
    printf("%c",a[i]);
    pcount=count;
}
.\n  ECHO;

```

```

%%
main()
{
    yylex();
    return 0;
}

```

Output:

```

nn@linuxmint ~ $ lex conv.l
nn@linuxmint ~ $ gcc lex.yy.c -lfl
nn@linuxmint ~ $ ./a.out<c.txt
DEEPAK A
F HUNAIF
NAVITHA 10
15 RAJINA
ABID 18
1A SANITHA
nn@linuxmint ~ $

```

// **c.txt**

```

DEEPAK 10
15 HUNAIF
NAVITHA 16
21 RAJINA
ABID 24
26 SANITHA

```

**Select Lines Ending with 'com' - Lex Program - Compiler Design
Program:**

```
// Lex file: com.l

%{
int count=0;
%}

%%
.*com\n {count++;ECHO;}
. ;

%%

main()
{
    yylex();
    printf("\nCount= %d\n",count);
    return 0;
}
```

Output:

```
nn@linuxmint ~ $ lex com.l
nn@linuxmint ~ $ gcc lex.yy.c -ll
nn@linuxmint ~ $ ./a.out<com.txt
```

```
www.google.com
www.yahoo.com
www.blogspot.com
```

```
Count= 3
nn@linuxmint ~ $
```

```
// com.txt
```

```
www.2k.tk
www.google.com
www.yahoo.com
```

Count of Words Starting with a - Lex Program - Compiler Design

Program:

```
// Lex file: aa.l
%{
    int count=0;
```



```
%}  
alpha  [a-zA-Z]  
digit  [0-9]  
space  [ \t\n]  
start  ^a  
%%
```

```
{start}                                {count++;}  
{space}{a|A}({alpha}|{digit})*  {count++;}  
.                                ;
```

```
%%
```

```
main()  
{  
    yylex();  
    printf("count= %d\n",count);  
}
```

Output:

```
nn@linuxmint ~ $ lex aa.l  
nn@linuxmint ~ $ gcc lex.yy.c -ll  
nn@linuxmint ~ $ ./a.out<tst.txt
```

```
count= 6  
nn@linuxmint ~ $  
// tst.txt  
afsal ARIFA aaa www.blogspot.com  
aiswarya saranya sooraj  
arun reshmi  
a www.cu.cc
```

LEX

set app. 3 / 3

Convert Lowercase to Uppercase & Reverse - Lex Program - Compiler Design

CHANGE CASE - LEX - PROGRAM

Program:

```
// Lex file: cap.l
lower [a-z]
CAPS [A-Z]
space [ \t\n]
%%
{lower}    {printf("%c",yytext[0]- 32);}
{CAPS}     {printf("%c",yytext[0]+ 32);}
{space}    ECHO;
.          ECHO;
%%
main()
{
```

```
yylex();
```

```
}
```

Output:

```
nn@linuxmint ~ $ lex cap.l
nn@linuxmint ~ $ gcc lex.yy.c -ll
nn@linuxmint ~ $ ./a.out<tst.txt
WWW.2K8618.BLOGSPOT.COM
sanjana jamsheena chaithanya neethu
GOVINDAPRASAD VIPIN ADARSH SHIVIN
baby brinda kavya helen
SALMAN TINU RICHARD SIBIN
SHIVIN laji NABEEL
www.2k8cse.cu.cc
nn@linuxmint ~ $
// tst.txt
www.2k8618.blogspot.com
SANJANA JAMSHEENA CHAITHANYA NEETHU
govindaprasad vipin adarsh shivin
BABY BRINDA KAVYA HELEN
salman tinu richard sabin
shivin LAJI nabeel
WWW.2K8CSE.CU.CC
```

<http://home.adelphi.edu/sbloch/class/271/examples/lexyacc/>

[ex1.1](#), copied out of the "flex" man page, replaces the word "username" wherever it appears in the input with the username of the person who's logged in right now; otherwise it copies stdin unaltered to stdout.

```
%%
username      printf("%s",getlogin());
```

[ex2.1](#), counts the number of lines and characters in the input file. Note that it uses the special symbol "." to match any character other than a newline.

```
int num_lines = 0, num_chars = 0;

%%
\n      ++num_lines; ++num_chars;
.       ++num_chars;

%%
int main ()
{
    yylex ();
    printf ("There were %d lines and %d characters.\n",
            num_lines, num_chars);
    return 0;
}
```

[caesar1.l](#) implements the Caesar cipher: it replaces every letter with the one three letters after in in alphabetical order, wrapping around at Z. It uses character ranges "[a-z]" and "[A-Z]", which match any lower-case letter and any upper-case letter respectively.

```
%%
[a-z]  { char ch = yytext[0];
        ch += 3;
        if (ch > 'z') ch -= ('z'+1-'a');
        printf ("%c", ch);
      }

[A-Z]  { char ch = yytext[0];
        ch += 3;
        if (ch > 'Z') ch -= ('Z'+1-'A');
        printf ("%c", ch);
      }
%%
```

[caesar2.l](#) does the same thing, but with a different (and briefer) approach: it uses range notation more cleverly to distinguish which characters need to wrap and which don't.

```
%%
[a-zA-W]      printf ("%c", yytext[0]+3);

[xyzXYZ]      printf ("%c", yytext[0]-23);
%%
```

[lexlongword.l](#) finds the longest word (defined as a contiguous string of upper and lower case letters) in the input. Although the range notation "[a-zA-Z]" would match any single letter, tacking on a "+" after it produces a regular expression matching any *sequence of one or more* letters, as long as possible (*i.e.* until it sees a non-letter). Compare with [clongword.c](#), [clongword2.c](#), and [clongword3.c](#).

```
/*lexlongword.l */

%{
#include <strings.h>

int longest = 0;
char longword[60];
%}

%%
[a-zA-Z]+      { if (yyleng > longest) {
                  longest = yyleng;
                  strcpy (longword, yytext);
                }
                }

.              |
\n             ;
%%
int main (void) {
  yylex ();
  printf ("The longest word was \"%s\\", which was %d characters long.\\n",
          longword, longest);
  return 0;
}
```



```
}
```

```
/* Reached end of file. Print the answer. */
```

```
printf ("The longest word was \"%s\", which was %d characters long.\n",  
        longword, longest);
```

```
return 0;
```

```
}
```

[clongword3.c](#).

```
#include <stdio.h>
```

```
#include <strings.h>
```

```
#define MAXLEN 60
```

```
int main (void) {
```

```
    char word[MAXLEN], longword[MAXLEN], pattern[10];
```

```
    int wordlen;
```

```
    int longest = 0;
```

```
    int ch;
```

```
    int result = 0;
```

```
sprintf (pattern, "%%%ds", MAXLEN);
```

```
while ((result = scanf (pattern, word)) != EOF) {
```

```
    if (strlen(word) > longest) {
```

```
        longest = strlen(word);
```

```
        strcpy (longword, word);
```

```
    }
```

```
}
```

```
/* Reached end of file. Print the answer. */
```

```
printf ("The longest word was \"%s\", which was %d characters long.\n",  
        longword, longest);
```

```
return 0;
```

```
}
```

[ch102.1](#), chapter 1 exercise 2 of the O'Reilly book, categorizes a number of words as verbs and nonverbs.

```
%{
```

```
/* this sample demonstrates very simple word recognition: verbs & other */
```

```
%}
```

```
%%
```

```
[\t ]+          /* ignore whitespace */ ;
```

```
is
```

```
am
```

```
are
```

```
were
```

```
was
```

```
be
```

```
being
```

```
been
```

```
do
```

```
does
```

```
did
```

```
will
```

```
would
```

```
should
```

```
can
```

```
could
```

```

has          |
have         |
had          |
go           { printf ("\'%s\' is a verb\n", yytext); }
[a-zA-Z]+    { printf ("\'%s\' is not a verb\n", yytext); }
.           |
\n          ECHO; /* which is the default anyway */
%%
int main (void) {
    return yylex ();
}
/* This is the "default main program" anyway, so it could be omitted. */

```

[ch103.1](#), also from the O'Reilly book, categorizes into verbs, adverbs, and other words. Still a very limited vocabulary.

```

%{
/* this sample demonstrates very simple word recognition: verbs & other */
%}
%%

[\\t ]+      /* ignore whitespace */ ;

is          |
am          |
are         |
were        |
was         |
be          |
being       |
been        |
do          |
does        |
did         |
will        |
would       |
should      |
can         |
could       |
has         |
have        |
had         |
go          { printf ("\'%s\' is a verb.\n", yytext); }

very |
simply |
gently |
quietly |
calmly |
angrily      { printf ("\'%s\' is an adverb.\n", yytext); }

[a-zA-Z]+    { printf ("\'%s\' is something else.\n", yytext); }
.           |
\n          ECHO; /* which is the default anyway */
%%
int main (void) {
    return yylex ();
}
/* This is the "default main program" anyway, so it could be omitted. */

```

[ex3.1](#), copied out of the "flex" man page, distinguishes keywords, integers, floats, identifiers, operators, and comments in a simple Pascal-like language. You don't need to understand everything that's going on in this one, but it demonstrates the kind of thing you can do fairly quickly and simply in lex.

```

/* scanner for a toy Pascal-like language */

%{
/* need this for the call to atof() below */
#include <math.h>
%}

DIGIT      [0-9]
ID         [a-z][a-z0-9]*

%%

{DIGIT}+   {
    printf( "An integer: %s (%d)\n", yytext,
            atoi( yytext ) );
}

{DIGIT}+"."{DIGIT}* {
    printf( "A float: %s (%g)\n", yytext,
            atof( yytext ) );
}

if|then|begin|end|procedure|function {
    printf( "A keyword: %s\n", yytext );
}

{ID}       printf( "An identifier: %s\n", yytext );

"+"|"-"|"*"|"/"  printf( "An operator: %s\n", yytext );

"{ "[^]\n}"      /* eat up one-line comments */

[ \t\n]+       /* eat up whitespace */

.              printf( "Unrecognized character: %s\n", yytext );

%%

int main( int argc, char **argv )
{
    ++argv, --argc; /* skip over program name */
    if ( argc > 0 )
        yyin = fopen( argv[0], "r" );
    else
        yyin = stdin;

    return yylex();
}

```

[romans.l](#) reads and interprets Roman numerals. Note how enormously much shorter and clearer this is than solving the same problem in C. This program depends on lex recognizing the *longest* of several alternative patterns, e.g. if it sees an I, it doesn't apply the action for the "I" rule until it has first checked that the next character isn't "V" or "X".

```

WS      [ \t]+

%%

        int total=0;

I        total += 1;
IV       total += 4;
V        total += 5;
IX       total += 9;
X        total += 10;
XL       total += 40;

```

```

L      total += 50;
XC     total += 90;
C      total += 100;
CD     total += 400;
D      total += 500;
CM     total += 900;
M      total += 1000;

{WS}   |
\n     return total;
%%
int main (void) {
    int first, second;

    first = yylex ();
    second = yylex ();

    printf ("%d + %d = %d\n", first, second, first+second);
    return 0;
}

```

[words.l](#) categorizes a variety of English words as nouns, verbs, adjectives, adverbs, etc. The main program doesn't do anything interesting with this information; it just prints out the sequence of parts of speech. A *real* English-language program would probably use this `flex` code to recognize parts of speech, combined with a `yacc` program to recognize the grammatical structure of the sentence.

```

        enum {ARTICLE, VERB, NOUN, ADJECTIVE,
              ADVERB, PUNCTUATION, PREPOSITION, END};
%%
<<EOF>>      return END;

" "          |
"\t"         ;

the          |
an?          return ARTICLE;

go(es)?      |
runs?        |
likes?       |
eats?        return VERB;

dogs?        |
cats?        |
fish         |
fox(es)?     |
glove        |
foxglove     |
moose        return NOUN;

quick        |
slow         |
lazy         |
clever       |
smart        |
stupid       |
brown        |
black        |
blue         |
red          |
orange       |
white        |
big          |
small        return ADJECTIVE;

quickly      |
easily      |

```


openly		
slowly		return ADVERB;
"."		
,"		
"?"		
;		
:		
!		
@		
#		
"\$"		
"%"		
"^"		
&		
"*"		
"("		
")"		return PUNCTUATION;

over		
under		
around		
through		
between		return PREPOSITION;

```

%%
int main(void) {
    int result;
    int running = 1;
    while (running) {
        result = yylex();
        switch (result) {
            case ARTICLE: printf("Article "); break;
            case VERB: printf("Verb "); break;
            case NOUN: printf("Noun "); break;
            case ADJECTIVE: printf("Adjective "); break;
            case ADVERB: printf("Adverb "); break;
            case PUNCTUATION: printf("Punctuation "); break;
            case PREPOSITION: printf("Preposition "); break;
            case END: running=0; break;
            default: printf ("Huh?");
        }
    }
    return 0;
}

```